Lab 4 – Data Structures

Muhammad Awais

25I-3207

Task 1:

Source Code:

```
#include <iostream>
using namespace std;

struct node{
	int data;
	node* next;
};

int count = 0;
int limit = 10;

bool isEmpty(node* head){
	if (!head)
	{
		cout << "List is empty!\n";
		return true;
	}
	return false;
}

bool isFull(){
	if (count >= limit)
	{
		cout << "List is full!\n";
		return true;
	}
	return false;
}

void insertFront(node*& head, int data){
	if (isFull()) return;
	node* newNode = new (nothrow) node;
	if (!newNode)
	{
		cout << "Memory allocation failed!\n";
		return;
	}
	newNode->data = data;
	newNode->next = head;
```

```
	head = newNode;
	count++;
}

void insertEnd(node*& head, int data){
	if (isFull()) return;
	node* newNode = new (nothrow) node;
	if (!newNode)
	{
		cout << "Memory allocation failed!\n";
		return;
	}
	newNode->data = data;
	newNode->next = nullptr;
	if (!head)
	{
		head = newNode;
		count++;
		return;
	}
	node* temp = head;
	while (temp->next) temp = temp->next;
	temp->next = newNode;
	count++;
}

void insertByNumber(node*& head, int afterValue, int data){
	if (isEmpty(head)) return;
	if (isFull()) return;
	node* temp = head;
	while (temp && temp->data != afterValue) temp = temp->next;
	if (!temp)
	{
		cout << "Value not found in list!\n";
		return;
	}
	node* newNode = new (nothrow) node;
	if (!newNode)
	{
		cout << "Memory allocation failed!\n";
		return;
	}
	newNode->data = data;
	newNode->next = temp->next;
```

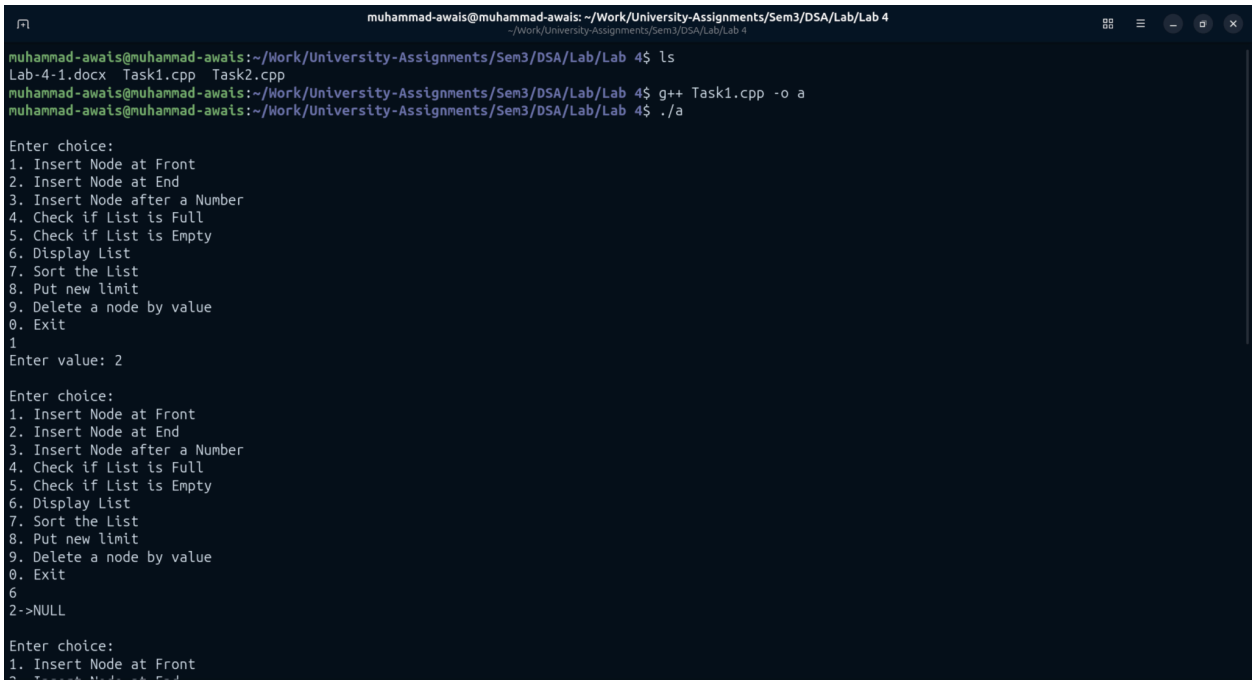
```
        temp->next = newNode;
        count++;
}

void display(node* head){
        if (isEmpty(head)) return;
        node* temp = head;
        while (temp)
        {
                cout << temp->data << "->";
                temp = temp->next;
        }
        cout << "NULL\n";
}

void sort(node* head){
        if (isEmpty(head)) return;
        node* outer = head;
        while (outer)
        {
                node* inner = outer->next;
                while(inner)
                {
                        if (inner->data < outer->data)
                        {
                                int temp = outer->data;
                                outer->data = inner->data;
                                inner->data = temp;
                        }
                        inner = inner->next;
                }
                outer = outer->next;
        }
}

void deleteNode(node*& head, int val){
        if (isEmpty(head)) return;
        if (head->data == val)
        {
                node* target = head;
                head = head->next;
                delete target;
                count--;
                return;
```

```
        }
        node* temp = head;
        while (temp->next && temp->next->data != val) temp = temp->next;
        if (!temp->next)
        {
                cout << "Value not found in list!\n";
                return;
        }
        node* target = temp->next;
        temp->next = target->next;
        delete target;
        count--;
}

void changeLimit(int newLimit){
        limit = newLimit;
        cout << "Limit updated to " << limit << ".\n";
}

int main(){
        node* head = nullptr;
        bool running = true;
        while (running)
        {
                cout << "\nEnter choice:\n1. Insert Node at Front"
                        << "\n2. Insert Node at End\n3. Insert Node after a Number\n"
                        << "4. Check if List is Full\n5. Check if List is Empty\n"
                        << "6. Display List\n7. Sort the List\n8. Put new limit\n"
                        << "9. Delete a node by value\n0. Exit\n";
                int choice;
                cin >> choice;
                switch (choice)
                {
                        case 0:
                                running = false;
                                break;
                        case 1:
                        {
                                int num;
                                cout << "Enter value: "; cin >> num;
                                insertFront(head, num);
                                break;
                        }
                        case 2:
```

```
        {
            int num;
            cout << "Enter value: "; cin >> num;
            insertEnd(head, num);
            break;
        }
        case 3:
        {
            int afterValue, num;
            cout << "Enter value to insert after: "; cin >> afterValue;
            cout << "Enter value to insert: "; cin >> num;
            insertByNumber(head, afterValue, num);
            break;
        }
        case 4:
            isFull();
            break;
        case 5:
            isEmpty(head);
            break;
        case 6:
            display(head);
            break;
        case 7:
            sort(head);
            cout << "List sorted.\n";
            break;
        case 8:
        {
            int newLimit;
            cout << "Enter new limit: "; cin >> newLimit;
            changeLimit(newLimit);
            break;
        }
        case 9:
        {
            int val;
            cout << "Enter value to delete: "; cin >> val;
            deleteNode(head, val);
            break;
        }
        default:
            cout << "Enter a valid choice!\n";
}
```

```
    }
    return 0;
}
```

Screenshot:



Task 2:

Source Code:

```
#include <iostream>
#include <string>
using namespace std;

struct Course
{
	string Code;
	string Name;
	int CHs;
	bool status;
	Course* next;
};

bool isEmpty(Course* path)
{
	if (!path)
	{
		cout << "No Academic Path specified!\n";
		return true;
	}
	return false;
}

void createAcademicPath(Course*& path, int& total)
{
	cout << "How many courses do you have: ";
	cin >> total;
	Course* tail = nullptr;
	for (int i = 0; i < total; i++)
	{
		Course* temp = new (nothrow) Course;
		if (!temp)
		{
			cout << "Memory allocation failed!\n";
			return;
		}
		cout << "Enter code of course " << i + 1 << ": "; cin >> temp->Code;
		cout << "Enter name of course " << i + 1 << ": "; cin >> temp->Name;
		cout << "Enter credit hours of course " << i + 1 << ": "; cin >> temp->CHs;
```

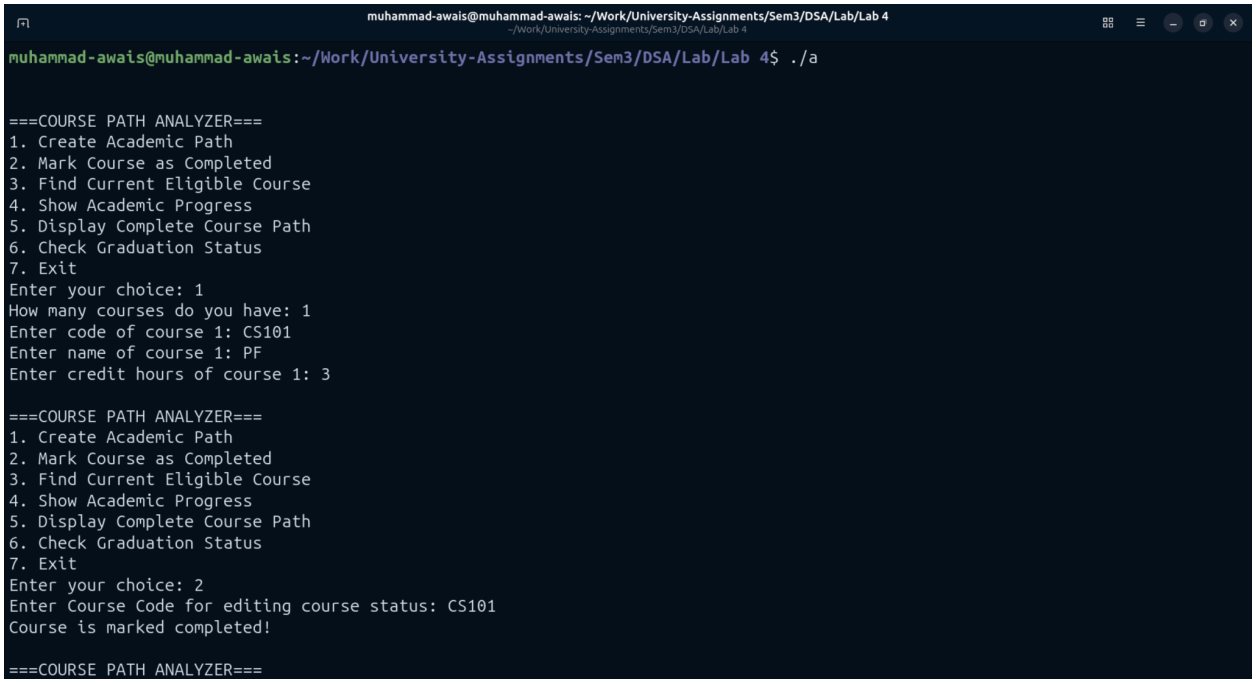
```
                temp->status = false;
                temp->next = nullptr;
                if (!path)
                {
                        path = temp;
                        tail = temp;
                }
                else
                {
                        tail->next = temp;
                        tail = temp;
                }
        }
}

void markCourseStatus(Course* path)
{
        if (isEmpty(path)) return;
        string code;
        cout << "Enter Course Code for editing course status: ";
        cin >> code;
        Course* temp = path;
        while (temp && temp->Code != code) temp = temp->next;
        if (!temp)
        {
                cout << "Course not found!\n";
                return;
        }
        if (temp->status)
        {
                cout << "Course already marked as completed!\n";
                return;
        }
        temp->status = true;
        cout << "Course is marked completed!\n";
}

void findEligibleCourse(Course* path)
{
        if (isEmpty(path)) return;
        Course* temp = path;
        while (temp && temp->status) temp = temp->next;
        if (!temp)
        {
```

```
                cout << "All courses completed, no course pending!\n";
                return;
        }
        cout << "You are currently eligible for:\n";
        cout << temp->Code << " - " << temp->Name << "\n";
}

void checkProgress(Course* path)
{
        if (isEmpty(path)) return;
        int completed = 0, total = 0;
        Course* temp = path;
        while (temp)
        {
                if (temp->status) completed++;
                total++;
                temp = temp->next;
        }
        int remaining = total - completed;
        double progress = (double)completed / total * 100;
        cout << "Courses Completed: " << completed << "\n";
        cout << "Courses Remaining: " << remaining << "\n";
        printf("Progress: %.0f%%\n", progress);
}

void displayPath(Course* path)
{
        if (isEmpty(path)) return;
        Course* temp = path;
        while (temp)
        {
                cout << temp->Code << " -> " << temp->Name << " -> " << (temp->status ?
"Completed" : "Not Completed") << "\n";
                temp = temp->next;
        }
}

void checkGraduation(Course* path)
{
        if (isEmpty(path)) return;
        int remaining = 0;
        Course* temp = path;
        while (temp)
        {
```

```
            if (!temp->status) remaining++;
            temp = temp->next;
        }
        if (!remaining)
        {
            cout << "All courses have been completed.\n";
            cout << "Student has completed the academic path.\n";
        }
        else
        {
            cout << "Student cannot complete the academic path yet.\n";
            cout << "Courses remaining: " << remaining << "\n";
        }
}

int main()
{
        Course* path = nullptr;
        int total = 0;
        bool running = true;
        while (running)
        {
            cout << "\n===COURSE PATH ANALYZER===\n";
            cout << "1. Create Academic Path\n2. Mark Course as Completed\n";
            cout << "3. Find Current Eligible Course\n4. Show Academic Progress\n";
            cout << "5. Display Complete Course Path\n6. Check Graduation Status\n";
            cout << "7. Exit\nEnter your choice: ";
            int choice;
            cin >> choice;
            switch (choice)
            {
                case 1:
                    createAcademicPath(path, total);
                    break;
                case 2:
                    markCourseStatus(path);
                    break;
                case 3:
                    findEligibleCourse(path);
                    break;
                case 4:
                    checkProgress(path);
                    break;
                case 5:
```

```
                displayPath(path);
                break;
            case 6:
                checkGraduation(path);
                break;
            case 7:
                running = false;
                break;
            default:
                cout << "Enter a valid choice!\n";
        }
    }
    return 0;
}
```

Screenshot:


muhammad-awais@muhammad-awais: ~/Work/University-Assignments/Sem3/DSA/Lab/Lab 4
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 4$ ./a
===COURSE PATH ANALYZER===
1. Create Academic Path
2. Mark Course as Completed
3. Find Current Eligible Course
4. Show Academic Progress
5. Display Complete Course Path
6. Check Graduation Status
7. Exit
Enter your choice: 1
How many courses do you have: 1
Enter code of course 1: CS101
Enter name of course 1: PF
Enter credit hours of course 1: 3
===COURSE PATH ANALYZER===
1. Create Academic Path
2. Mark Course as Completed
3. Find Current Eligible Course
4. Show Academic Progress
5. Display Complete Course Path
6. Check Graduation Status
7. Exit
Enter your choice: 2
Enter Course Code for editing course status: CS101
Course is marked completed!
===COURSE PATH ANALYZER===


muhammad-awais@muhammad-awais: ~/Work/University-Assignments/Sem3/DSA/Lab/Lab 4
===COURSE PATH ANALYZER===
1. Create Academic Path
2. Mark Course as Completed
3. Find Current Eligible Course
4. Show Academic Progress
5. Display Complete Course Path
6. Check Graduation Status
7. Exit
Enter your choice: 4
Courses Completed: 1
Courses Remaining: 0
Progress: 100%
===COURSE PATH ANALYZER===
1. Create Academic Path
2. Mark Course as Completed
3. Find Current Eligible Course
4. Show Academic Progress
5. Display Complete Course Path
6. Check Graduation Status
7. Exit
Enter your choice: 5
CS101 -> PF -> Completed
===COURSE PATH ANALYZER===
1. Create Academic Path
2. Mark Course as Completed
3. Find Current Eligible Course
4. Show Academic Progress
5. Display Complete Course Path
6. Check Graduation Status
7. Exit
Enter your choice: 7
muhammad-awais@muhammad-awais:~/Work/University-Assignments/Sem3/DSA/Lab/Lab 4$